Aligning architecture to Unit Economics

Every technical design decision carries a direct financial consequence. A design without grounding choices in unit economics risks building systems that are operationally unsustainable, unprofitable, or misaligned with business strategy.

Unit revenue must scale faster than unit infrastructure costs. If a platform's cost to serve grows linearly with transaction volume, high adoption can reduce profitability. Tracking unit costs can lead to inefficient API calls, unoptimized database queries, 3P dependencies eating up profit margins and can prevent unexpected budget overruns. Product teams can determine which are expensive features, what features should go to which consumption tier.

Note that executives evaluate technical solutions based on ROI, Payback period, EBITDA impact. It is important that the architects speak the vocabulary of unit economics for strategic business enablement. For example, "Refactoring this messaging architecture reduces cloud compute cost per transaction by 40% expanding gross margins from 65% to 78%" will be more effective than saying "Refactoring would result in cleaner and maintainable code".

Cost to Serve (COGS) - Measures direct infrastructure and support cost to run a single tenant; An architect should focus on Resource utilization, Auto-scaling policy, 3P API costs.
Average Revenue Per User (ARPU) - Defines expected revenue yield per user; An architect should focus on System isolation level, SLA guarantees, Multi-region replication options.
Customer Acquisition Cost (CAC) - Pushes reducing customer onboarding costs; An architect should focus on automating tenant provisioning, DevOps pipelines.
LTV:CAC Ratio (LTV means LifeTime Value of the customer) - Indicates total financial return per customer over time; An architect should focus on building resilient, low latency architecture preventing any possible customer churn.

In this case study, let us look at how a business generates revenue and how operational metrics shape up the profitability.

Case Study: TransactCloud Modernization & Enterprise Scale

Company Context: TransactCloud is a B2B SaaS platform providing core payment orchestration and fraud prevention workflows for mid-market e-commerce merchants. The company is transitioning from a single-tenant VM model to a cloud-native, multi-tenant serverless architecture.

Let us dive deep on how business generates revenue …

TransactCloud operates a hybrid monetization model combining predictable recurring SaaS subscriptions with transaction volume usage fees:

Total Revenue = Fixed SaaS Subscription + Variable Usage Fee
= (Base Platform Fee) + (Transaction Volume x Take Rate)

Illustration:

- **Fixed Subscription Fee:** $2,000/month per merchant enterprise tier for access to core management dashboards, API access, and baseline analytics.
  Variable Transaction Fee: $0.05 per processed transaction + 0.10% take rate on total Gross Merchandise Value (GMV).
- **Average Account Profile:** A typical mid-market merchant processes 100,000 transactions/month with a total GMV of $5,000,000.
  - **Monthly Revenue per Merchant:** $2,000 (Base) + $5,000 ($0.05 x 100k) + $5,000 (0.10% x $5M) = **$12,000 MRR** ($144,000 ARR).

To sustain operations and scale revenue, usually the capital is allocated into the following buckets. Understanding this split is important to identify the areas for cost optimization.

| Sales & Marketing (40%) | Cloud & Infra (25%) | R&D / Engg (20%) | General & Administrative (15%) |

Now, let us look into Key operational metrics that drive executive priorities. Executive leadership evaluates all engineering and business decisions against 4 foundational unit economic drivers - CAC, LTV, EBITDA, Customer Churn Rate.

A. Customer Acquisition Cost (CAC)

- **Definition:** The total sales and marketing expenditure required to land one new merchant account.
- **Current Baseline:** $36,000 per acquired enterprise merchant.
- **Executive Priority:** Lower CAC payback period from 6 months down to 3 months by accelerating onboarding pipelines.

B. Lifetime Value (LTV)

- **Definition:** Total gross profit a single customer generates throughout their lifecycle before churning.
- **Current Baseline:** Assuming an average merchant lifetime of 48 months at an 80% Gross Margin:

LTV = (ARPU * Gross Margin %) * Customer LIfe Span

LTV = $12000/month x 80% x 48 months = $460800

Let us understand GMV (Gross Margin Value) first. Why is it important? It measures ecosystem liquidity; A rising GMV indicates expanding network; service is attracting more customers. It helps in capacity planning, forces the evaluation of transaction throughput, cloud capacity, database scaling needs, compute scaling needs, and like. GMV demonstrates market opportunity.

GMV = Total number of transaction x Average Order Value (AOV)

Example: If an e-commerce marketplace processes 100,000 orders in a month at an average order value of $50, the platform's GMV for that month is: GMV is 100000 x $50 = $50000; Note that GMV is not revenue, Revenue is GMV x Take Rate (Say, 10% commission fee on market place); In such case, Revenue is $50000 x 10% = $5000

Gross Margin % is ((Total Revenue - COGS)/(Total Revenue))*100

Note: Cost of Goods Sold (COGS) include Cloud Infrastructure and Hosting, 3P Software, Customer Support, DevOps, Engineers' Salary, Software licensing costs - No indirect costs or OpEx.

Consider a cloud software company evaluating its quarterly performance:

- **Total Quarterly Revenue:** $10,000,000
- **Direct Delivery Costs (COGS):**
  - Cloud Hosting (AWS): $1,200,000
  - Third-Party Security & API Services: $300,000
  - Customer Onboarding & Technical Support Salaries: $500,000
  - **Total COGS: $2,000,000**

Gross Margin % works out to be **80%.**

Now that we have LTV and CAC, let us deduce LTV:CAC Ratio. That is, $460800:$36000

LTV is 12.8 times CAC is healthy unit economics. This is what executive leadership would be interested in.

C. EBITDA & Gross Margin Expansion

- **Definition:** Earnings Before Interest, Taxes, Depreciation, and Amortization—reflecting true operational profitability.
- **Executive Priority:** The executive team wants to expand overall Gross Margin from **65% to 80%** by modernizing infrastructure.
- **Engineering Impact:** Moving from single-tenant dedicated VMs to a multi-tenant auto-scaling cluster cuts cloud infrastructure cost per transaction from $0.025 down to $0.008, directly adding $1.7M annually back to EBITDA.

D. Customer Churn Rate

- **Definition:** Percentage of customer ARR lost over a specific period.
- **Current Baseline:** Monthly account churn is 0.8% (~9.6% annualized).
- **Executive Priority:** Reduce churn to <0.5% monthly. The primary cause of churn is transaction processing latency spikes during peak sales events (e.g., Black Friday).

Note: LTV can be calculated with monthly customer churn rate as well.

LTV = (ARPU * Gross Margin %) / Monthly Customer Churn Rate

Now that we have financial literacy that drives the business, let us summarize an example of architecture alignment.

Metric	**Technical focus area**	**Architecture strategy**	**Business outcome**
EBITDA / Gross Margin	1\ HIgh cloud compute costs 2\ Idle instances 3\ Dedicated VMs	1\ Re-architect to multi-tenancy solution 2\ Serverless options 3\ Containers 4\ Auto scaling	Infrastructure cost per transaction drops to 68%, pushing Gross margin from 65% to 80%
Customer Churn	1\ Unplanned downtime during peak transaction windows	1\ Multi-region deployments 2\ Cell based architecture 3\ Fail-over strategies	Platform availability increases to 99.99%, reducing customer churn by 30%
CAC Payback	1\ Manual security provisioning delays onboarding by 6 weeks	1\ Tenant provisioning using IaC	Onboarding drops from 6 weeks to 10 minutes accelerating time to first transaction revenue.

As a Solutions Architect, you now can align Architecture priorities with Unit economics.